

Scientific Python — Quick Reference

Python, NumPy, Matplotlib, SciPy, and SymPy for physics and scientific computing — with everything the Statistics book uses

1. Python Basics

Variables — A name refers to an object.

```
x = 3
y = 2.5
z = x + y

print(x)
print(z)
```

Output: 3
5.5

Basic types

```
a = 3
b = 3.14
c = 2 + 3j
d = "hello"
e = True
f = None

print(type(a))
print(type(c))
print(type(d))
print(type(e))
print(type(f))
```

print() — Display values.

```
x = 3.14
print(x)
print("x =", x)
```

Output: 3.14
x = 3.14

type() — Find the type of an object.

```
x = 3
print(type(x))
```

Output: <class 'int'>

isinstance() — Test whether an object has a given type.

```
x = 3.0

print(isinstance(x, float))
print(isinstance(x, int))
```

Output: True
False

2. Operators

Arithmetic

```
print(7 + 2)
print(7 - 2)
print(7 * 2)
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(7 ** 2)
```

Output: 9
5
14
3.5
3
1
49

Comparison

```
x = 3

print(x == 3)
print(x != 3)
print(x > 2)
print(x >= 3)
print(x < 5)
print(x <= 2)
```

Output: True
False
True
True
True
False

Logical operators

```
x = 3

print(x > 0 and x < 5)
print(x < 0 or x > 2)
print(not (x > 0))
```

Output: True
True
False

in — Test membership.

```
x = [1, 2, 3]

print(2 in x)
print(5 in x)
```

Output: True
False

3. Strings

Strings — Text enclosed by quotes.

```
name = "Chandra"

print(name)
print(name[0])
print(name[-1])
```

Output: Chandra
C
a

String slicing

```
s = "abcdef"

print(s[1:4])
print(s[:3])
print(s[3:])
print(s[::-1])
```

Output: bcd
abc
def
fedcba

Useful string methods

```
s = " Hello Python "
```

```
print(s.strip())
print(s.lower())
print(s.upper())
print(s.replace("Python", "World"))
```

f-strings — Insert values directly into text.

```
x = 3.14159

print(f"x = {x}")
print(f"x = {x:.2f}")
```

Output: x = 3.14159
x = 3.14

4. Conditions and Loops

if / elif / else — Conditional logic.

```
x = -2

if x > 0:
    print("positive")
elif x < 0:
    print("negative")
else:
    print("zero")
```

Output: negative

for — Iterate over elements.

```
for i in range(5):
    print(i)
```

Output: 0 1 2 3 4

range() — Generate integer sequences.

```
print(list(range(5)))
print(list(range(2, 7)))
print(list(range(0, 10, 2)))
```

Output: [0, 1, 2, 3, 4]
[2, 3, 4, 5, 6]
[0, 2, 4, 6, 8]

while — Repeat while a condition is true.

```
i = 0

while i < 3:
    print(i)
    i += 1
```

Output: 0 1 2

break — Exit a loop immediately.

```
for i in range(10):
    if i == 3:
        break
    print(i)
```

Output: 0 1 2

continue — Skip the current iteration.

```
for i in range(5):
    if i == 2:
        continue
    print(i)
```

Output: 0 1 3 4

pass — Do nothing; useful as a placeholder.

```
for i in range(3):
    pass

print("done")
```

Output: done

5. Built-in Functions

len() — Number of elements.

```
x = [2, 4, 6]

print(len(x))
```

Output: 3

sum() — Sum elements.

```
x = [1, 2, 3]

print(sum(x))
```

Output: 6

min() / **max()** — Smallest/largest value.

```
x = [4, 2, 9, 1]

print(min(x))
print(max(x))
```

Output: 1
9

abs() — Absolute value.

```
print(abs(-3.5))
```

Output: 3.5

sorted() — Return a sorted copy.

```
x = [3, 1, 2]

print(sorted(x))
print(x)
```

Output: [1, 2, 3]
[3, 1, 2]

6. Containers

List — Ordered, mutable collection.

```
x = [1, 2, 3]

x.append(4)
print(x)
```

Output: [1, 2, 3, 4]

List methods

```
x = [1, 2, 3]

x.append(4)
x.extend([5, 6])
x.insert(0, 0)

print(x)
```

Output: [0, 1, 2, 3, 4, 5, 6]

pop() — Remove and return an element.

```
x = [10, 20, 30]

y = x.pop()

print(y)
print(x)
```

Output: 30
[10, 20]

remove() — Remove a specific value.

```
x = [1, 2, 3, 2]

x.remove(2)

print(x)
```

Output: [1, 3, 2]

sort() — Sort a list in place.

```
x = [3, 1, 2]

x.sort()

print(x)
```

Output: [1, 2, 3]

Tuple — Ordered, immutable collection.

```
r = (1, 2, 3)

print(r)
print(r[0])
```

Output: (1, 2, 3)
1

Useful for fixed quantities such as coordinates.

Unpacking — Assign several values at once.

```
x = (10, 20, 30)

a, b, c = x

print(a)
print(b)
print(c)
```

Output: 10
20
30

Dictionary — Key-value storage.

```
particle = {
    "mass": 2.0,
    "charge": 1.0
}

print(particle["mass"])
print(particle["charge"])
```

Output: 2.0
1.0

Dictionary methods

```
d = {"a": 1, "b": 2}

print(d.keys())
print(d.values())
print(d.items())
print(d.get("c", 0))
```

`get(key, default)` avoids an error if the key is absent.

Loop over a dictionary

```
d = {"m": 2, "q": 1}

for key, value in d.items():
    print(key, value)
```

Output: m 2
q 1

Set — Unordered collection of unique values.

```
x = {1, 2, 2, 3}

print(x)
print(2 in x)
```

Output: {1, 2, 3}
True

7. Indexing and Slicing

Indexing

```
x = [10, 20, 30, 40]

print(x[0])
print(x[-1])
```

Output: 10
40
Python starts indexing at zero.

Slicing

```
x = [0, 1, 2, 3, 4, 5]

print(x[1:4])
print(x[:3])
print(x[3:])
print(x[:2])
print(x[::2])
print(x[::-1])
```

Output: [1, 2, 3]
[0, 1, 2]
[3, 4, 5]
[0, 2, 4]
[5, 4, 3, 2, 1, 0]

8. enumerate() and zip()

enumerate() — Loop over index and value.

```
x = [10, 20, 30]

for i, value in enumerate(x):
    print(i, value)
```

Output: 0 10
1 20
2 30

zip() — Iterate over sequences together.

```
x = [1, 2, 3]
y = [4, 5, 6]

for a, b in zip(x, y):
    print(a, b)
```

Output: 1 4
2 5
3 6

9. Comprehensions

List comprehension — Compact loop construction.

```
x = [i**2 for i in range(5)]

print(x)
```

Output: [0, 1, 4, 9, 16]

Conditional comprehension

```
x = [i for i in range(10) if i % 2 == 0]

print(x)
```

Output: [0, 2, 4, 6, 8]

10. Functions

def — Define a function.

```
def energy(m, v):
    return 0.5 * m * v**2

E = energy(2, 3)

print(E)
```

Output: 9.0

return — Send a value out of a function.

```
def square(x):
    y = x**2
    return y

print(square(4))
```

Output: 16

Multiple return values

```
def powers(x):
    return x**2, x**3

a, b = powers(2)

print(a)
print(b)
```

Output: 4
8

Default arguments

```
def greet(name, message="hello"):
    print(message, name)

greet("Chandra")
greet("Chandra", "hi")
```

Output: hello Chandra
hi Chandra

lambda — Small anonymous function.

```
f = lambda x: x**2

print(f(3))
```

Output: 9
Often used by SciPy functions.

11. References and Copying

Assignment does not copy a list.

```
a = [1, 2, 3]
b = a

b[0] = 100

print(a)
print(b)
```

Output: [100, 2, 3]
[100, 2, 3]
Both names refer to the same list.

copy() — Make an independent copy.

```
a = [1, 2, 3]
b = a.copy()

b[0] = 100

print(a)
print(b)
```

Output: [1, 2, 3]
[100, 2, 3]

is — Test object identity.

```
a = [1, 2]
b = a
c = a.copy()

print(a is b)
print(a is c)
```

Output: True
False

12. Modules and Imports

import — Load a module.

```
import math

print(math.sqrt(16))
print(math.pi)
```

Output: 4.0
3.141592653589793

Import selected functions

```
from math import sqrt, pi

print(sqrt(16))
print(pi)
```

Alias a module

```
import numpy as np

x = np.array([1, 2, 3])

print(x)
```

Output: [1 2 3]

13. Useful Inspection Tools

dir() — See attributes and methods.

```
x = [1, 2, 3]

print(dir(x))
```

Very useful when you forget a method.

help() — Read built-in documentation.

```
help(str.split)
```

Useful at the interactive prompt.

None — Represents “no value”.

```
x = None

print(x)
print(x is None)
```

Output: None
True

14. Errors and Exceptions

try / except — Handle an error.

```
try:
    x = 1 / 0
except ZeroDivisionError:
    print("division by zero")
```

Output: division by zero

raise — Explicitly generate an error.

```
def energy(m, v):
    if m <= 0:
        raise ValueError("mass must be positive")
    return 0.5 * m * v**2
```

assert — Check an assumption.

```
mass = 2.0

assert mass > 0

print("valid")
```

Output: valid
Useful for numerical sanity checks.

15. Files

open() — Open a file.

```
with open("data.txt", "r") as f:
    text = f.read()

print(text)
```

"r" = read.

Write a file

```
with open("output.txt", "w") as f:
    f.write("hello\n")
    f.write("world\n")
```

"w" = write and overwrite.

Read lines

```
with open("data.txt", "r") as f:
    lines = f.readlines()

print(lines)
```

with — Automatically closes the file.

```
with open("data.txt") as f:
    data = f.read()
```

Prefer this to manually calling `close()`.

16. NumPy Essentials

np.array() — Create numerical arrays.

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])

print(x)
print(type(x))
```

Output: [1. 2. 3.]
<class 'numpy.ndarray'>

np.zeros() — Array of zeros.

```
x = np.zeros(5)

print(x)
```

Output: [0. 0. 0. 0. 0.]

np.ones() — Array of ones.

```
x = np.ones(4)

print(x)
```

Output: [1. 1. 1. 1.]

np.eye() — Identity matrix.

```
I = np.eye(3)

print(I)
```

np.linspace() — Evenly spaced points.

```
x = np.linspace(0, 1, 5)

print(x)
```

Output: [0. 0.25 0.5 0.75 1.]
Very common for plotting.

np.arange() — Numerical range.

```
x = np.arange(0, 1, 0.2)

print(x)
```

Output: [0. 0.2 0.4 0.6 0.8]

shape — Array dimensions.

```
A = np.zeros((3, 4))

print(A.shape)
print(A.ndim)
print(A.size)
```

Output: (3, 4)

2

12

reshape() — Change array shape.

```
x = np.arange(12)
A = x.reshape(3, 4)

print(A)
```

dtype — Numerical data type.

```
x = np.array([1, 2, 3])

print(x.dtype)
```

17. NumPy Indexing and Slicing

Array indexing

```
x = np.array([10, 20, 30, 40])

print(x[0])
print(x[-1])
```

Output: 10
40

Array slicing

```
x = np.arange(6)

print(x[1:4])
print(x[::2])
```

Output: [1 2 3]
[0 2 4]

Matrix indexing

```
A = np.array([[1, 2],
              [3, 4]])

print(A[0, 1])
print(A[:, 0])
print(A[1, :])
```

Output: 2
[1 3]
[3 4]

18. Vectorization and Broadcasting

Elementwise operations

```
x = np.array([1, 2, 3])

print(2 * x)
print(x**2)
print(x + 10)
```

Output: [2 4 6]
[1 4 9]
[11 12 13]

Broadcasting — Compatible shapes can interact without explicit loops.

```
x = np.array([1, 2, 3])

y = np.array([10, 20, 30])

print(x + y)
print(2 * x + y)
```

Output: [11 22 33]
[12 24 36]

Boolean masking

```
x = np.array([-2, 1, 4, -1])

print(x > 0)
print(x[x > 0])
```

Output: [False True True False]
[1 4]

np.where() — Conditional selection.

```
x = np.array([-1, 2, -3, 4])

y = np.where(x > 0, x, 0)

print(y)
```

Output: [0 2 0 4]

np.all() / **np.any()**

```
x = np.array([1, 2, 3])

print(np.all(x > 0))
print(np.any(x > 2))
```

Output: True
True

19. NumPy Numerical Tools

Elementary functions

```
x = np.array([0, 1, 2])

print(np.sqrt(x))
print(np.exp(x))
print(np.sin(x))
```

np.sum() / **np.mean()**

```
x = np.array([1, 2, 3, 4])

print(np.sum(x))
print(np.mean(x))
```

Output: 10
2.5

np.std() / **np.var()**

```
x = np.array([1, 2, 3, 4])

print(np.std(x))
print(np.var(x))
```

np.argmax() / **np.argmin()** — Index of maximum/minimum.

```
x = np.array([4, 9, 2, 7])

print(np.argmax(x))
print(np.argmin(x))
```

Output: 1
2

np.diff() — Neighboring differences.

```
x = np.array([1, 4, 9, 16])

print(np.diff(x))
```

Output: [3 5 7]

np.gradient() — Numerical derivative.

```
t = np.linspace(0, 10, 1000)
x = np.sin(t)

v = np.gradient(x, t)
```

Approximates

$$v = \frac{dx}{dt}.$$

np.isclose() — Compare floating-point values.

```
print(np.isclose(0.1 + 0.2, 0.3))
```

Output: True
Prefer this to == for floating-point comparisons.

np.allclose() — Compare arrays.

```
A = np.array([1.0, 2.0])
B = np.array([1.0, 2.0000001])

print(np.allclose(A, B))
```

Output: True

`np.isnan() / np.isfinite()`

```
x = np.array([1.0, np.nan, np.inf])  
  
print(np.isnan(x))  
print(np.isfinite(x))
```

20. Linear Algebra

`np.dot()` — Dot product.

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])  
  
print(np.dot(a, b))
```

Output: 32

$$\mathbf{a} \cdot \mathbf{b}.$$

`@` — Matrix multiplication.

```
A = np.array([[1, 2],  
              [3, 4]])  
  
B = np.array([[5, 6],  
              [7, 8]])  
  
print(A @ B)
```

Output:

$$\begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

`.T` — Transpose.

```
A = np.array([[1, 2],  
              [3, 4]])  
  
print(A.T)
```

`np.cross()` — Cross product.

```
r = np.array([1, 0, 0])  
p = np.array([0, 2, 0])  
  
L = np.cross(r, p)  
  
print(L)
```

Output: [0 0 2]

$$\mathbf{L} = \mathbf{r} \times \mathbf{p}.$$

`np.linalg.norm()` — Vector magnitude.

```
r = np.array([3, 4])  
  
print(np.linalg.norm(r))
```

Output: 5.0

Unit vector

```
r_hat = r / np.linalg.norm(r)  
  
print(r_hat)
```

$$\hat{\mathbf{r}} = \frac{\mathbf{r}}{|\mathbf{r}|}.$$

21. More Linear Algebra

`np.linalg.solve()` — Solve

$$Ax = b.$$

```
A = np.array([[2, 1],
              [1, 3]])

b = np.array([5, 6])

x = np.linalg.solve(A, b)

print(x)
```

`np.linalg.eig()` — Eigenvalues/eigenvectors.

```
A = np.array([[2, 0],
              [0, 3]])

w, V = np.linalg.eig(A)

print(w)
print(V)
```

$$Av = \lambda v.$$

`np.linalg.det()` — Determinant.

```
A = np.array([[1, 2],
              [3, 4]])

print(np.linalg.det(A))
```

Output: -2.0

22. Matplotlib

`plt.plot()` — Line plot.

```
import matplotlib.pyplot as plt

t = np.linspace(0, 10, 1000)
x = np.cos(t)

plt.plot(t, x)
plt.show()
```

Labels and title

```
plt.xlabel("Time")
plt.ylabel("Position")
plt.title("Oscillator")
```

`plt.legend()`

```
plt.plot(t, np.cos(t), label="cos")
plt.plot(t, np.sin(t), label="sin")

plt.legend()
plt.show()
```

`plt.grid()`

```
plt.plot(t, x)

plt.grid()
plt.show()
```

`plt.scatter()` — Scatter plot.

```
plt.scatter(t, x)
plt.show()
```

Useful for data points.

`plt.subplots()` — Explicit figure and axes.

```
fig, ax = plt.subplots()

ax.plot(t, x)
ax.set_xlabel("t")
ax.set_ylabel("x")

plt.show()
```

Preferred for larger programs.

Logarithmic axes

```
plt.loglog(x, y)
plt.show()
```

Also: `plt.semilogx()` and `plt.semilogy()`.

`plt.savefig()` — Save a figure.

```
plt.savefig("spectrum.png", dpi=300)
```

23. SciPy

`solve_ivp()` — Solve an ODE.

```
from scipy.integrate import solve_ivp

def rhs(t, y):
    x, v = y
    return [v, -x]

sol = solve_ivp(
    rhs,
    [0, 10],
    [1, 0]
)

print(sol.t.shape)
print(sol.y.shape)
```

The state is

$$\mathbf{y} = \begin{pmatrix} x \\ v \end{pmatrix}.$$

`quad()` — Numerical integration.

```
from scipy.integrate import quad

f = lambda x: x**2

I, err = quad(f, 0, 1)

print(I)
print(err)
```

`root_scalar()` — Find a scalar root.

```
from scipy.optimize import root_scalar

f = lambda x: x**2 - 2

result = root_scalar(
    f,
    bracket=[0, 2]
)

print(result.root)
```

Output is approximately: 1.41421356

`minimize()` — Numerical optimization.

```
from scipy.optimize import minimize

f = lambda x: (x - 3)**2

result = minimize(f, 0)

print(result.x)
```

Output is approximately: [3.]

curve_fit() — Fit a model to data.

```
from scipy.optimize import curve_fit

def model(x, a, b):
    return a*x + b

popt, pcov = curve_fit(
    model, x, y
)

print(popt)
```

popt contains fitted parameters. **pcov** contains their covariance estimate.

24. SymPy

symbols() — Create symbolic variables.

```
import sympy as sp

x = sp.symbols("x")

print(x)
```

Output: **x**

diff() — Symbolic differentiation.

```
f = x**3 + 2*x

print(sp.diff(f, x))
```

Output:

$$3x^2 + 2.$$

integrate() — Symbolic integration.

```
print(
    sp.integrate(x**2, x)
)
```

Output:

$$\frac{x^3}{3}.$$

solve() — Solve equations symbolically.

```
print(
    sp.solve(x**2 - 4, x)
)
```

Output: **[-2, 2]**

simplify() — Simplify expressions.

```
expr = (x**2 - 1)/(x - 1)

print(sp.simplify(expr))
```

Result:

$$x + 1.$$

series() — Taylor expansion.

```
print(
    sp.series(sp.sin(x), x, 0, 5)
)
```

Output:

$$x - \frac{x^3}{6} + O(x^5).$$

25. Useful Scientific Patterns

Typical physics workflow

```
import numpy as np
import matplotlib.pyplot as plt

t = np.linspace(0, 10, 1000)
x = np.cos(t)

plt.plot(t, x)
plt.xlabel("t")
plt.ylabel("x(t)")
plt.show()
```

Parameter sweep

```
for alpha in [0.1, 0.2, 0.5]:
    result = simulate(alpha)
    print(alpha, result)
```

Useful for scanning model parameters.

Numerical derivative

```
v = np.gradient(x, t)
a = np.gradient(v, t)

print(v)
print(a)
```

Corresponds to

$$v = \dot{x}, \quad a = \ddot{x}.$$

Conservation check

```
E = kinetic + potential

print(np.max(E))
print(np.min(E))
print(
    np.max(E) - np.min(E)
)
```

The last quantity measures the numerical variation of E .

Position, velocity, acceleration

```
r = np.array([x, y, z])
v = np.array([vx, vy, vz])

speed = np.linalg.norm(v)

print(speed)
```

$$|\mathbf{v}| = \sqrt{\mathbf{v} \cdot \mathbf{v}}.$$

Angular momentum

```
L = np.cross(r, p)

print(L)
```

$$\mathbf{L} = \mathbf{r} \times \mathbf{p}.$$

Energy

```
K = 0.5 * m * np.dot(v, v)
U = potential(r)

E = K + U

print(K)
print(U)
print(E)
```

$$E = K + U.$$

ODE state vector

For

$$\ddot{x} = f(x, \dot{x}, t),$$

use

$$\mathbf{y} = \begin{pmatrix} x \\ \dot{x} \end{pmatrix}.$$

```
def rhs(t, y):
    x, v = y
    return [v, force(x, v, t)]
```

26. Random Numbers

Modern random-number generator

```
rng = np.random.default_rng(123)

x = rng.random(5)

print(x)
```

A seed makes the sequence reproducible.

Gaussian random numbers

```
rng = np.random.default_rng(123)

x = rng.normal(
    loc=0,
    scale=1,
    size=5
)

print(x)
```

Useful for simulations and Monte Carlo methods.

27. Script Structure

```
if __name__ == "__main__"
```

```
def main():
    print("running program")

if __name__ == "__main__":
    main()
```

The block runs when the file is executed directly, not when it is imported as a module.

Typical reusable script

```
import numpy as np

def model(x):
    return np.exp(-x**2)

def main():
    x = np.linspace(-5, 5, 1000)
    y = model(x)
    print(y[0])

if __name__ == "__main__":
    main()
```

28. What to Memorize First

- Variables, numbers, strings, booleans, `None`
- `+` `-` `*` `/` `//` `%` `**`
- `==` `!=` `<` `>` `<=` `>=`
- `and`, `or`, `not`, `in`
- `if` / `elif` / `else`
- `for`, `while`
- `break`, `continue`
- Lists, tuples, dictionaries, sets
- Indexing and slicing
- `enumerate()`, `zip()`
- Functions, arguments, `return`
- List comprehensions
- References vs. copies
- `import`
- `try` / `except`, `assert`
- File handling with `with` `open()`
- NumPy arrays and `shape`
- Vectorization and broadcasting
- `dot()`, `cross()`, `norm()`
- Matplotlib plotting
- SciPy ODE solving, integration, optimization
- SymPy differentiation and solving

The Rest of What the Book Uses

Sections 29–41: everything the book’s scripts call that sections 1–28 do not cover

29. The Top of Every Script

pathlib.Path — Paths that work on every computer.

```
from pathlib import Path

here = Path(__file__).resolve()
print(here.name)      # this file
print(here.parent)    # its folder
print(here.parents[1]) # one folder up
```

resolve() turns a relative path into a full one.

sys.path — Let Python find the book’s shared code.

```
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from common import setup, savefig, save_numbers
```

Adds code/ to the import search, so **common.py** can be imported from any chapter folder.

Files on disk

```
out = Path("data/ch06")
out.mkdir(parents=True, exist_ok=True)

f = out / "samples.npz"
print(f.exists())
```

The **/** operator joins path pieces.

30. Converting Types

int(), **float()**, **str()**

```
print(int(3.9))
print(int("42"))
print(float("1e-3"))
print(str(3.0) + " m")
```

Output: 3

42

0.001

3.0 m

int() cuts towards zero; it does not round.

round()

```
print(round(3.14159, 2))
print(round(2.675, 2))
```

Output: 3.14

2.67

2.675 is stored as 2.67499... in binary, so it rounds down.

.astype() — Convert a whole array.

```
x = np.array([1.7, -1.7])

print(x.astype(int))
```

Output: [1 -1]

dict.update() — Add or overwrite several keys.

```
d = {"a": 1, "b": 2}
d.update({"b": 3, "c": 4})

print(d)
```

Output: {'a': 1, 'b': 3, 'c': 4}

31. Matplotlib the Book’s Way: fig, ax

One figure, one axes — every call goes to **ax**.

```
fig, ax = plt.subplots(figsize=(5, 3.5))

ax.plot(x, y, label="model")
ax.set_title("Power spectrum")
ax.set_xlim(0, 10)
ax.set_ylim(-1, 1)
ax.legend()

fig.tight_layout()
fig.savefig("spectrum.pdf")
```

tight_layout() stops labels from overlapping.

Several panels

```
fig, axes = plt.subplots(1, 2, figsize=(8, 3))

axes[0].plot(x, y)
axes[1].hist(samples, bins=40)
```

axes is an array of axes, one per panel.

Reference lines and text

```
ax.axhline(0, color="gray", lw=0.8)
ax.axvline(1.96, ls="--")
ax.text(0.05, 0.9, "p = 0.05",
       transform=ax.transAxes)
```

With **transform=ax.transAxes** the position is a fraction of the panel, (0,0) bottom left.

Histograms and error bars

```
ax.hist(samples, bins=50, density=True)

ax.errorbar(x, y, yerr=sigma, fmt="o",
           capsize=2)
```

density=True makes the bars integrate to one, comparable with a pdf.

Bands, scales, images

```
ax.fill_between(x, lo, hi, alpha=0.3)

ax.set_xscale("log")
ax.set_yscale("log")

im = ax.imshow(Z, origin="lower")
fig.colorbar(im, ax=ax)
ax.set_aspect("equal")
```

fill.between draws error bands such as $\pm 1\sigma$.

32. Building Arrays

np.full(), **np.empty()**, **np.zeros_like()**

```
print(np.full(3, 7.0))

a = np.empty(1000) # filled later
b = np.zeros_like(a) # same shape, zeros
```

Output: [7. 7. 7.]

np.empty contains leftover garbage until you fill it.

np.logspace() / **np.geomspace()** — Evenly spaced in log.

```
print(np.geomspace(1, 1000, 4))
print(np.logspace(0, 3, 4))
```

Both give 1, 10, 100, 1000: **geomspace** takes the end values, **logspace** their powers of ten.

np.diag()

```
C = np.diag([1.0, 4.0]) # matrix from diagonal
print(np.diag(C))      # diagonal from matrix
```

Output: [1. 4.]

Error bars from a covariance: `np.sqrt(np.diag(C))`.

Joining arrays

```
a = np.array([1, 2])
b = np.array([3, 4])

print(np.concatenate([a, b]))
print(np.column_stack([a, b]))
```

Output: [1 2 3 4]

[[1 3]

[2 4]]

Also `np.vstack` (rows) and `np.stack` (new axis).

.ravel() — Flatten to one dimension.

```
A = np.array([[1, 2], [3, 4]])

print(A.ravel())
```

Output: [1 2 3 4]

np.meshgrid() — A 2D grid of coordinates.

```
x = np.linspace(-1, 1, 200)
y = np.linspace(-1, 1, 100)
X, Y = np.meshgrid(x, y)

R = np.sqrt(X**2 + Y**2)
print(R.shape)
```

Output: (100, 200)

Rows follow *y*, columns follow *x*.

33. More NumPy Maths

Logs, powers, absolute values

```
print(np.log(np.e))
print(np.log10(1000))
print(np.abs(3 + 4j))
```

Output: 1.0

3.0

5.0

`np.log` is the natural log, *ln*.

np.cumsum(), np.maximum(), np.clip()

```
print(np.cumsum([1, 2, 3]))
print(np.maximum([1, 5], [3, 2]))
print(np.clip([-2, 0.5, 3], 0, 1))
```

Output: [1 3 6]

[3 5]

[0. 0.5 1.]

`np.maximum` compares element by element; `np.max` gives one number.

np.median(), np.percentile(), np.quantile()

```
x = rng.standard_normal(100000)

print(np.median(x))
print(np.percentile(x, [16, 84]))
print(np.quantile(x, 0.975))
```

Close to 0, $[-1, 1]$ and 1.96: the 68% interval and a one-sided 97.5% point of a Gaussian.

np.interp() — Straight-line interpolation.

```
print(np.interp(2.5, [1, 2, 3], [10, 20, 30]))
```

Output: 25.0

x must be increasing.

np.trapezoid() — Integrate sampled values.

```
x = np.linspace(0, 1, 101)

print(np.trapezoid(x**2, x))
```

Output: about 0.33335, against the exact $1/3$.

Called `np.trapz` before NumPy 2.0. Use `quad` when you have the function, `trapezoid` when you only have samples.

Counting: np.histogram(), np.bincount()

```
counts, edges = np.histogram(x, bins=20)
print(len(counts), len(edges))

print(np.bincount([0, 1, 1, 3]))
```

Output: 20 21

[1 2 0 1]

N bins have *N* + 1 edges.

Sorting and searching

```
x = np.array([30, 10, 20])

print(np.sort(x))
print(np.argsort(x))
print(np.searchsorted([1, 3, 5], 4))
print(np.unique([3, 1, 3, 2]))
```

Output: [10 20 30]

[1 2 0]

2

[1 2 3]

np.cov() — Sample covariance matrix.

```
C = np.cov(x, y) # 2 x 2
print(C[0, 1])  # covariance of x and y
```

Divides by *N* − 1. Rows are variables.

np.outer() and np.einsum()

```
print(np.outer([1, 2], [3, 4]))

chi2 = np.einsum("i,ij,j->", r, Cinv, r)
```

Output: [[3 4]

[6 8]]

The `einsum` line is $\chi^2 = \mathbf{r}^T \mathbf{C}^{-1} \mathbf{r}$: repeated letters are summed.

np.polyfit() — Least-squares polynomial.

```
slope, intercept = np.polyfit(x, y, 1)
```

Highest power first.

34. More Random Numbers

The generator's other methods

```
rng = np.random.default_rng(42)

z = rng.standard_normal(1000) # N(0, 1)
u = rng.uniform(0, 1, size=1000)
n = rng.poisson(4.0, size=1000) # counts
k = rng.integers(0, 10, size=5) # 0..9
```

`integers(0, 10)` never returns 10.

choice() — Draw from a list (resampling).

```
boot = rng.choice(data, size=len(data),
                  replace=True)
```

One bootstrap sample. `replace=False` shuffles without repeats.

multivariate.normal() — Correlated Gaussians.

```
mean = [0, 0]
C = [[1.0, 0.8],
      [0.8, 1.0]]

xy = rng.multivariate_normal(mean, C, size=5000)
print(xy.shape)
```

Output: (5000, 2)

35. Saving and Loading Arrays

np.savez() / **np.load()**

```
np.savez("run.npz", ell=ell, cl=c1)

d = np.load("run.npz")
print(d["cl"][:3])
```

Several named arrays in one file, read back exactly.

np.loadtxt() — Read a table of numbers.

```
z, H, sigma = np.loadtxt("chronometers.txt",
                          unpack=True)
```

`unpack=True` gives one array per column. Lines starting with `#` are skipped.

36. Linear Algebra for Statistics

solve, not inv

```
x = np.linalg.solve(C, b)    # good
x = np.linalg.inv(C) @ b     # same, less accurate
```

Use `inv` only when the inverse itself is wanted, e.g. the error matrix F^{-1} from a Fisher matrix F .

np.linalg.cholesky() — $C = LL^T$.

```
L = np.linalg.cholesky(C)

z = rng.standard_normal(len(C))
x = L @ z    # covariance C

logdet = 2 * np.sum(np.log(np.diag(L)))
```

Correlated noise and $\ln \det C$ in one step. Fails if C is not positive definite: a useful warning.

np.linalg.eigh() — Symmetric matrices.

```
w, V = np.linalg.eigh(C)

print(w)    # ascending
print(w.max() / w.min())
```

The last line is the condition number: about how many times errors in $\mathbf{b}$ can be magnified when solving $C\mathbf{x} = \mathbf{b}$.

37. Fourier Transforms

np.fft.rfft() — Spectrum of a real signal.

```
n, dt = 1024, 0.01
t = np.arange(n) * dt
x = np.sin(2 * np.pi * 5 * t)

X = np.fft.rfft(x)
f = np.fft.rfftfreq(n, dt)

print(f[np.argmax(np.abs(X))])
```

Output: about 4.98, the nearest frequency bin to the 5 Hz input.

Power: `np.abs(X)**2`. Back to time: `np.fft.irfft(X, n)`.

38. Probability Distributions: `scipy.stats`

The four questions to ask a distribution

```
from scipy import stats

print(stats.norm.pdf(0))    # density
print(stats.norm.cdf(1.96)) # P(X <= x)
print(stats.norm.sf(5))     # P(X > x)
print(stats.norm.ppf(0.975)) # inverse cdf
```

Output: 0.3989...

0.9750...

2.8665...e-07

1.9599...

`sf = 1 - cdf`, but accurate far in the tail (the 5σ p -value).

Other distributions, same methods

```
print(stats.chi2.sf(30, df=20))    # p-value
print(stats.chi2.ppf(0.95, df=1))
print(stats.poisson.pmf(3, 2.0))  # P(N = 3)

x = stats.norm.rvs(loc=1, scale=2,
                    size=5, random_state=rng)
```

Output (first three): about 0.07, 3.84, 0.180.

Discrete distributions have `pmf` instead of `pdf`.

39. Roots and Special Functions

brentq() — The book's root finder.

```
from scipy.optimize import brentq

r = brentq(lambda x: x**2 - 2, 0, 2)

print(r)
```

Output: 1.414213562373...

f must change sign between the two ends.

gammaln() / **logsumexp()** — Work in logs.

```
from scipy.special import gammaln, logsumexp

print(gammaln(101))    # ln(100!)
print(logsumexp([1000, 1000]))
```

Output: 363.739...

1000.693...

100! and e^{1000} overflow; their logarithms do not.

40. Timing

time.perf_counter()

```
import time

t0 = time.perf_counter()
result = expensive(n)
print(f"{time.perf_counter() - t0:.3f} s")
```

Time a few small n , fit the slope of $\log t$ against $\log n$, and extrapolate before launching a long run.

41. Black Boxes: `healpy` and `CAMB`

healpy — Maps on the sphere.

```
import healpy as hp

nside = 64
print(hp.nside2npix(nside))

alm = hp.map2alm(m, lmax=128) # map -> a_lm
cl = hp.alm2cl(alm)           # a_lm -> C_l
m2 = hp.alm2map(alm, nside)  # a_lm -> map
cl2 = hp.anafast(m, lmax=128) # map -> C_l
hp.mollview(m)
```

Output (first line): 49152, that is $12 N_{\text{side}}^2$ pixels.

CAMB — Theory spectra from parameters.

```
import camb

pars = camb.set_params(H0=67.4, ombh2=0.0224,
                      omch2=0.120, As=2.1e-9,
                      ns=0.965, lmax=2500)
res = camb.get_results(pars)
cl = res.get_cmb_power_spectra(pars,
                               CMB_unit="muK", raw_cl=True)["total"][:, 0]
```

Six numbers in, C_ℓ^{TT} out. In the book this is wrapped in `camb.fiducial.py`.

The book's own helpers (`code/common.py`)

- `setup(w, h)`: house plot style and size
- `savefig(fig, "ch06", "name")`: the figure in `figures/ch06/`
- `save_numbers("ch06", "name", {...})`: numbers as L^AT_EX macros quoted in the text
- `rng_for("ch06", "name")`: a seeded generator for that script, the same on every computer